Clumpy Medium Extension of LaRT: Algorithm Description

Contents

1. Overview **2**

2. Physical Setup **2**

2.1 Geometry 2

2.2 Clump Count 2

2.3 Clump Physical Parameters 3

2.4 Clump Velocities 3

3. Clump Placement: Random Sequential Addition (RSA) **4**

3.1 Algorithm 4

3.2 Acceptance Rate 5

4. CSR Acceleration Grid **5**

4.1 Structure 5

4.2 Construction (Two-Pass) 6

5. Raytrace Through the Clump Medium **6**

5.1 Overview 6

5.2 Ray–Sphere Intersection 6

5.3 DDA Search for Next Clump 6

5.4 Optical Depth Traversal 7

5.5 Optical Depth to Edge 8

6. Peel-Off Observations **8**

7. FITS Output of Clump Information **8**

8. Input Parameters Summary **10**

9. Memory Layout **11**

10. Validation **11**

1. Overview

The clumpy medium mode adds a population of N non-overlapping spherical clumps inside a spherical domain of radius R. Each clump is a sphere of radius r_c with uniform Lyman-α opacity; the inter-clump medium is vacuum.

The mode is activated by

```
par%use_clump_medium = .true.
```

in the input namelist. All raytrace procedure pointers are redirected to clump-aware routines (`raytrace_clump.f90`) while scatter and peel-off reuse the standard Cartesian routines from `scattering_car.f90` and `peelingoff_rect.f90`, since the uniform clump physical parameters (T, $v_{\rm bulk}$) are broadcast to the Cartesian grid arrays (`grid%Dfreq`, `grid%voigt_a`).

New source files introduced for this mode:

File	Role
`clump_mod.f90`	Clump population: init, RSA placement, CSR grid, raytrace helpers
`raytrace_clump.f90`	Raytrace through the clump medium (DDA + sphere intersection)
`grid_mod_clump.f90`	Grid setup: creates Cartesian shell grid, then calls `init_clumps`

2. Physical Setup

2.1 Geometry

The outer domain is a sphere of radius R = `par%rmax` (code units). N spherical clumps of radius r_c = `par%clump_radius` are placed at random inside this sphere (non-overlapping).

A Cartesian grid of $N_x \times N_y \times N_z$ cells is created with the same bounding box $[-R, R]^3$. Grid arrays (`rhokap`, `Dfreq`, `voigt_a`, `vf`) are set to uniform values:

- `rhokap` $= 0$ everywhere (opacity lives only inside clumps);
- `Dfreq` $= \Delta\nu_D^{cl}$ (clump Doppler frequency);
- `voigt_a` $= a_V^{cl}$ (clump Voigt parameter);
- `vfx` $=$ `vfy` $=$ `vfz` $= 0$.

Peel-off and scattering routines read these uniform grid values, so no special-casing is needed inside those modules.

2.2 Clump Count

The user specifies one of three equivalent parameters; the code derives N from whichever is set:

From $N_{\rm cov}$ (covering factor): The mean number of clumps intercepted by a radial ray from the centre to the sphere surface is

$$N_{\rm cov} = N\,\frac{\pi r_c^2}{\pi R^2}\cdot\frac{R}{\frac{4}{3}\pi R^3/(4\pi R^2)} = \frac{3}{4}\,N\Big(\frac{r_c}{R}\Big)^2, \tag{1}$$

hence

$$N = \frac{4}{3}\,N_{\rm cov}\left(\frac{R}{r_c}\right)^2. \tag{2}$$

Input parameter: `par%clump_N_cov`.

From $f_{\rm vol}$ (volume filling factor):

$$f_{\rm vol} = N\left(\frac{r_c}{R}\right)^3, \qquad N = f_{\rm vol}\left(\frac{R}{r_c}\right)^3. \tag{3}$$

Input parameter: `par%clump_f_vol`.

Direct specification: `par%clump_N_clumps`.

2.3 Clump Physical Parameters

The clump temperature T_c defaults to `par%temperature` unless `par%clump_temperature` > 0. Derived quantities:

$$v_{\rm th} = v_{\rm th,1}\sqrt{T_c}, \quad v_{\rm th,1} = \sqrt{2k_B/m_H} \approx 12.84\ {\rm km\,s^{-1}}/\sqrt{10^4\,{\rm K}}, \tag{4}$$

$$\Delta\nu_D^{cl} = v_{\rm th}/(\lambda_0 \times {\rm km2um}), \tag{5}$$

$$a_V^{cl} = \frac{A_{21}}{4\pi\,\Delta\nu_D^{cl}}, \tag{6}$$

where λ_0 is the Lyman-α rest wavelength.

The line-centre optical depth of a clump (centre to surface) is `par%clump_tau0`; from this the uniform opacity is

$$\kappa_c = \frac{\tau_0}{\phi(0, a_V^{cl})\,r_c}, \tag{7}$$

where $\phi(x, a)$ is the Voigt profile at dimensionless frequency x. Alternatively, the hydrogen number density `par%clump_nH` may be given, in which case $\kappa_c = n_H\,\sigma_0\,d_{\rm cm}$ (requires `par%distance_unit`).

2.4 Clump Velocities

Each clump's velocity is the sum of two independent components:

$$\mathbf{v}_i = \mathbf{v}_{\rm sys}(\mathbf{r}_i) + \mathbf{v}_{\rm rand,i}, \tag{8}$$

where $\mathbf{v}_{\rm sys}$ is a position-dependent systematic flow (set by `par%velocity_type`) and $\mathbf{v}_{\rm rand}$ is an independent Gaussian random component (set by `par%clump_sigma_v`). Either component can be zero.

Random component: If `par%clump_sigma_v` > 0, each clump is assigned $\mathbf{v}_{\mathrm{rand},i} \sim \mathcal{N}(0, \sigma_v^2)$ per component (drawn in `generate_clumps`).

Systematic component (`par%velocity_type`): After RSA placement, `assign_clump_velocities_from_type` evaluates the flow velocity at each clump centre and *adds* it to the existing (possibly non-zero) velocity. Supported types:

`velocity_type`	Formula	Parameters
`'hubble'`	$\mathbf{v} = V_{\mathrm{exp}}\,\mathbf{r}/R$	`Vexp`
`'constant_radial'`	$\mathbf{v} = V_{\mathrm{exp}}\,\hat{\mathbf{r}}$	`Vexp`
`'parallel_velocity'`	$\mathbf{v} = (V_x, V_y, V_z)$	`Vx, Vy, Vz`
`'ssh'`	see below	`Vpeak, rpeak, DeltaV`
`'rotating_solid_body'`	$\mathbf{v} = V_{\mathrm{rot}}(-y, x, 0)/R$	`Vrot`
`'rotating_galaxy_halo'`	flat rotation about z	`Vrot, rinner`

All velocity magnitudes are in km/s. The `'ssh'` model (Song, Seon & Hwang 2020) gives a linear rise inside r_{peak} and a Hubble-like outer flow:

$$v(r) = \begin{cases} V_{\mathrm{peak}}\, r/r_{\mathrm{peak}} & r < r_{\mathrm{peak}}, \\ V_{\mathrm{peak}} + \Delta V\,(r - r_{\mathrm{peak}})/(R - r_{\mathrm{peak}}) & r \geq r_{\mathrm{peak}}. \end{cases} \tag{9}$$

The `'rotating_galaxy_halo'` model uses the cylindrical radius $\rho = \sqrt{x^2 + y^2}$ and gives solid-body rotation inside r_{inner} and a flat curve ($v_\phi = V_{\mathrm{rot}}$) outside.

3. Clump Placement: Random Sequential Addition (RSA)

3.1 Algorithm

Clumps are placed one at a time. Each candidate position is drawn uniformly inside the sphere (box-rejection sampling) and accepted only if it does not overlap any previously placed clump (centre separation $\geq 2r_c$).

Overlap checking is accelerated by a linked-list hash grid of cell size $\ell_{\mathrm{RSA}} \approx 2R/N_g$ where $N_g = \min(512, \lfloor N^{1/3} \rfloor + 1)$. Only the $3^3 = 27$ neighbouring cells of the candidate are checked, reducing the expected check count to $O(1)$ per attempt (for $f_{\mathrm{vol}} < 35\%$).

```
do while (icl < N_clumps)
   !--- draw uniform point inside sphere
   do
      xc = (2*rand - 1)*R; yc = ...; zc = ...
      if (xc^2+yc^2+zc^2 <= R^2) exit
```

```
   enddo
   !--- check 27 neighbours for overlap (min_sep = 2*r_c)
   overlap = .false.
   outer: do kg2=...; do jg2=...; do ig2=...
      jnb = head(cell)
      do while (jnb > 0)
         if (dist2(xc,yc,zc, cl_x(jnb),...) < (2*r_c)^2) then
            overlap=.true.; exit outer
         endif
         jnb = nxt(jnb)
      enddo
   enddo; enddo; enddo outer
   if (overlap) cycle
   icl = icl+1
   cl_x(icl)=xc; cl_y(icl)=yc; cl_z(icl)=zc
   ! assign velocity if sigma_v > 0
   ! insert into linked list
enddo
```

3.2 Acceptance Rate

For the test case ($N_{\rm cov} = 5$, $r_c/R = 0.01$, $N = 66667$, $f_{\rm vol} = 0.067$) the RSA acceptance rate is approximately 74%.

4. CSR Acceleration Grid

After RSA placement, a compressed-sparse-row (CSR) grid is built to accelerate ray–clump intersection queries during radiative transfer.

4.1 Structure

A 3-D Cartesian grid of $M_x \times M_y \times M_z$ cells ($M = N_g$ from above) covers the domain $[-R-r_c, R+r_c]^3$. Each clump is registered in every CSR cell whose bounding box overlaps the clump sphere. The CSR layout uses two arrays:

- `cg_start(icell)` – index into `cg_list` of the first entry for cell `icell`;
- `cg_list(j)` – clump index stored at position j.

4.2 Construction (Two-Pass)

Pass 1: count registrations per cell → prefix-sum to fill `cg_start`.

Pass 2: fill `cg_list`.

A clump is registered in cell (i, j, k) if the cell's AABB intersects the clump sphere, i.e. the cell range in each dimension is $[\lfloor (c_x - r_c - x_{\min})/\delta \rfloor, \lfloor (c_x + r_c - x_{\min})/\delta \rfloor]$.

Both `cg_start` and `cg_list` are allocated as MPI-3 shared-memory windows (one copy per node).

5. Raytrace Through the Clump Medium

5.1 Overview

Photon propagation is split into two phases:

1. **Advance through vacuum** – use the DDA traversal of the CSR grid (§5.3) to find the next clump hit.
2. **Traverse through a clump** – accumulate optical depth along a straight path through the sphere (§5.4).

Frequency shifts due to individual clump bulk velocities are applied at clump entry and reversed at clump exit.

5.2 Ray–Sphere Intersection

Given a ray $\mathbf{p}(t) = \mathbf{o} + t\hat{\mathbf{k}}$ and a sphere centred at $\mathbf{c}$ with squared radius r_c^2, define $\mathbf{r} = \mathbf{o} - \mathbf{c}$, $b = \mathbf{r} \cdot \hat{\mathbf{k}}$, $\Delta = b^2 - |\mathbf{r}|^2 + r_c^2$. If $\Delta < 0$ there is no intersection. Otherwise

$$t_{\text{entry}} = -b - \sqrt{\Delta}, \qquad t_{\text{exit}} = -b + \sqrt{\Delta}. \tag{10}$$

A hit is recorded when $t_{\text{exit}} > 0$.

5.3 DDA Search for Next Clump

The Amanatides & Woo (1987) digital differential analyser (DDA) traverses the CSR cells along the ray in order of increasing path length. For each cell, all registered clumps are tested with the ray–sphere formula. The traversal stops as soon as the current cell's entry distance exceeds the best hit distance found so far.

```
best_te   = huge;  best_icl = 0
d         = 0     ! path length to current cell entry

```

```
do while (.true.)
   if (d > best_te .or. d > t_max) exit
   ! test all clumps in current cell
   do ip = cg_start(icell), cg_start(icell+1)-1
      icl = cg_list(ip)
      if (icl == skip_icl) cycle
      call ray_sphere_isect(..., te, tx2, hit)
      if (hit .and. tx2>0 .and. te<best_te) then
         best_te=te; best_tx2=tx2; best_icl=icl
      endif
   enddo
   ! advance to next cell (Amanatides & Woo)
   if (tx<=ty .and. tx<=tz) then
      d=tx; ci=ci+si; tx=tx+delx
   elseif (ty<=tz) then
      d=ty; cj=cj+sj; ty=ty+dely
   else
      d=tz; ck=ck+sk; tz=tz+delz
   endif
enddo
```

5.4 Optical Depth Traversal

`raytrace_to_tau_clump` advances the photon until optical depth $\tau_{\rm in}$ is accumulated or the outer sphere is exited.

Inside a clump (`icell_clump` > 0): The remaining path through the current clump is $s = t_{\rm exit}(\mathbf{p}, \hat{\mathbf{k}}, {\rm icl})$. Opacity at the current frequency: $\kappa = \kappa_c\,\phi(x, a_V)$. If $\tau_{\rm rem} \le \kappa s$, scatter at $\mathrm{d}s = \tau_{\rm rem}/\kappa$. Otherwise subtract κs from $\tau_{\rm rem}$, advance to clump exit, shift frequency back to the global frame, and continue in vacuum.

In vacuum (`icell_clump` $= 0$): Call `find_next_clump` (DDA, §5.3) to find the nearest clump. If none is found along the ray, advance to the sphere boundary and exit (`photon%inside` = `.false.`).

Frequency convention: The photon frequency x (in units of $\Delta\nu_D^{cl}$) is maintained in the local clump rest frame while inside a clump, and in the global frame in vacuum:

$$\text{at clump entry:} \quad x \leftarrow x - u_{\rm los}/v_{\rm th}, \tag{11}$$

$$\text{at clump exit:} \quad x \leftarrow x + u_{\rm los}/v_{\rm th}, \tag{12}$$

where $u_{\rm los} = (\mathbf{v}_i \cdot \hat{\mathbf{k}})$.

Jout accumulation: When the photon exits the outer sphere (`photon%inside` becomes `.false.`), the photon weight is accumulated into the escape spectrum:

$$J_{\rm out}[i_x] \mathrel{+}= w, \qquad i_x = \left\lfloor \frac{x - x_{\rm min}}{\delta x} \right\rfloor + 1, \tag{13}$$

where x is the photon frequency (global frame, in units of $\Delta\nu_D$) and w is the photon weight. This mirrors the behaviour of `raytrace_to_tau_car`, where Jout is accumulated upon grid exit. There are three exit paths (clump-then-sphere exit, already-outside-sphere, and no-clump-found), all handled identically.

5.5 Optical Depth to Edge

`raytrace_to_edge_clump` computes τ from the current position to the sphere boundary without moving the photon (read-only input). The same DDA logic is used; frequency shifts at clump entry/exit are tracked locally. This is called by peel-off routines and by the forced-first-scatter initialisation in `run_simulation_mod`.

6. Peel-Off Observations

Peel-off is handled by the standard `peelingoff_rect` routines (`peeling_direct_outside`, `peeling_resonance_nostokes` etc.) with `raytrace_to_edge` pointing to `raytrace_to_edge_clump`.
The observer distance defaults to $d = 100\,R$ and the image pixel size is set so that the full sphere subtends the image:

$$\delta\theta = \frac{1}{N_x/2} \arcsin\left(\frac{R}{d}\right) \text{ [rad]} \approx \frac{R}{d\,(N_x/2)} \text{ [rad]}. \tag{14}$$

For $R = 1$, $d = 100$, $N_x = 129$: $\delta\theta \approx 00089$, covering ± 114, which exactly spans the sphere.

7. FITS Output of Clump Information

When `par%save_clump_info = .true.`, the subroutine `write_clumps_fits` (in `clump_mod.f90`) writes a binary-table FITS file `<base>_clumps.fits.gz` containing:

BinTable columns: `X`, `Y`, `Z` (clump centres, code units); `VX`, `VY`, `VZ` (bulk velocities, km/s).

Header keywords (in the BinTable HDU):

Keyword	Value	Description
`N_CLUMPS`	N	number of clumps (realised)
`SPHERE_R`	R	outer sphere radius (code units)
`CL_RAD`	r_c	clump radius (code units)
`F_VOL`	f_{vol}	volume filling factor (realised)
`N_COV`	N_{cov}	covering factor (realised)
`TAU0`	τ_0	line-centre τ per clump
`SIGMA_V`	σ_v	bulk velocity dispersion (km/s)
`TEMP_CL`	T_c	clump temperature used (K)
`RHOKAP`	κ_c	opacity per code unit
`CL_DFREQ`	$\Delta\nu_D^{cl}$	Doppler frequency (Hz)
`VTHERM`	v_{th}	thermal velocity (km/s)
`VOIGT_A`	a_V	Voigt damping parameter
`RMAX`	R	`par%rmax` (input)
`IN_NCOV`	`par%clump_N_cov`	input covering factor
`IN_FVOL`	`par%clump_f_vol`	input filling factor
`IN_NCL`	`par%clump_N_clumps`	input N
`IN_NH`	`par%clump_nH`	input n_H (cm^{-3})
`IN_TEMP`	`par%clump_temperature`	input T_c (K)

Note: column data are written first (which creates the BinTable HDU); header keywords are then written into that same HDU. FITS keyword names must not exceed 8 characters.

8. Input Parameters Summary

Parameter	Default	Description
`par%use_clump_medium`	`.false.`	enable clump mode
`par%rmax`	—	outer sphere radius (code units)
`par%clump_radius`	—	clump radius r_c (code units)
`par%clump_N_cov`	0	covering factor (input)
`par%clump_f_vol`	0	volume filling factor (input)
`par%clump_N_clumps`	0	number of clumps (input, direct)
`par%clump_tau0`	0	line-centre τ per clump
`par%clump_nH`	0	clump n_H (cm^{-3}; requires `distance_unit`)
`par%clump_temperature`	-1	clump T (K); $< 0 \Rightarrow$ `par%temperature`
`par%clump_sigma_v`	0	random velocity σ per component (km/s)
`par%velocity_type`	’’	systematic flow type (see §2.4)
`par%Vexp`	0	expansion speed for `hubble`/`constant_radial` (km/s)
`par%Vx, Vy, Vz`	0	bulk velocity for `parallel_velocity` (km/s)
`par%Vpeak`	0	peak speed for `ssh` (km/s)
`par%rpeak`	0	peak radius for `ssh` (code units)
`par%DeltaV`	0	outer velocity increment for `ssh` (km/s)
`par%Vrot`	0	rotation speed for rotating models (km/s)
`par%rinner`	0	solid-body radius for `rotating_galaxy_halo`
`par%save_clump_info`	`.false.`	write clump FITS file

Example input:

```
&parameters
 par%use_clump_medium = .true.
 par%rmax             = 1.0
 par%clump_radius     = 0.01
 par%clump_N_cov      = 5.0
 par%clump_tau0       = 1000.0
 par%temperature      = 1.0e4
 par%clump_sigma_v    = 0.0
 par%no_photons       = 1e5
 par%spectral_type    = 'voigt'
 par%nxfreq           = 301
 par%velocity_min     = -100.0
 par%velocity_max     =  100.0
 par%nx = 11, par%ny = 11, par%nz = 11
 par%nxim = 129, par%nyim = 129
```

```
 par%save_clump_info   = .true.
 par%out_file          = 'clump_test.fits.gz'
/
```

9. Memory Layout

Clump positions, velocities, and CSR arrays are allocated as MPI-3 shared-memory windows via `create_shared_mem` from `memory_mod`, giving one copy per compute node rather than per MPI rank. On systems with multiple ranks per node this significantly reduces memory usage for large clump populations.
Cleanup is performed by `destroy_clumps` which calls `destroy_shared_mem_all` (collective `MPI_WIN_FREE` across all ranks) followed by explicit `nullify` of all pointers. The individual `destroy_mem` is not used because `MPI_WIN_SHARED_QUERY` returns the rank-0 base address, which differs from the local pointer on non-root ranks.

10. Validation

The test case `examples/clump_sphere/clump_test.in` sets $R = 1$, $r_c = 0.01$, $N_{\rm cov} = 5$, $\tau_0 = 1000$, $T = 10^4\,\mathrm{K}$, 10^5 photons, 10 MPI ranks. Typical results:

- Realised $N_{\rm cov} \approx 5.00$, $f_{\rm vol} \approx 0.067$;
- RSA acceptance rate $\sim 74\%$, $N = 66667$ clumps placed;
- Mean scattering count ≈ 6400 per photon;
- Wall time ≈ 3.75 min (10 ranks).

The escaped spectrum $J_{\rm out}(x)$ shows the characteristic double-peak expected for a clumpy Lyman-α source. The observer peel-off (`_obs.fits.gz`) contains both `Scattered` (HDU 0) and `Direct` (HDU 1) spectral image cubes of shape $(301, 129, 129)$.